

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO2 – Develop a Computer Program (Simple)

Nominal Duration: Theory 3.5 Hours / Practical 12 Hours

LEARNING OUTCOME 02

Develop a Computer Program (Simple)

Upon completion of this Learning Outcome, students/trainees will have the following competencies:

1. Analyze a simple programming problem.
2. Identify the requirements of a programming problem.
3. Identify the required input, processing and output.
4. Open Python IDLE or an appropriate Python development environment.
5. Explain the basic structure of a Python program.
6. Apply Python syntax and indentation rules.
7. Use comments to document Python programs.
8. Use variables to store basic data.
9. Use the `print()` function to display output.
10. Write simple Python programs according to given requirements.
11. Save Python programs using the `.py` extension.
12. Execute Python programs using Python interpreter.
13. Test Python programs using appropriate test data.
14. Identify syntax, runtime and logical errors.
15. Apply basic debugging techniques.
16. Correct errors in Python programs.
17. Verify program output against requirements.

INFORMATION SHEET No. 2.1

Title: Fundamentals of Developing a Simple Python Program

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain the basic structure of a Python program.
- Explain the programming problem-solving process.
- Identify input, processing and output.
- Explain Python syntax and indentation.
- Explain Python comments.
- Use variables for basic data storage.
- Use `print()` to display information.
- Explain Python Shell and Script Mode.
- Explain common Python errors.
- Apply basic debugging techniques.
- Explain program testing and verification.

1. Understanding a Programming Problem

Before writing a program, a programmer must understand the problem.

A simple programming problem normally contains:

1. **Problem**
2. **Input**
3. **Processing**
4. **Output**
5. **Requirements**

For example:

Develop a program that calculates the area of a rectangle.

The programmer should identify:

Input:

- Length
- Width

Processing:

Area = Length × Width

Output:

- Area of rectangle

2. Problem Analysis

Problem analysis means studying the problem before writing code.

A programmer should ask:

- What information is given?
- What information is required?
- What calculations are needed?

- What output is required?
- What conditions must the program follow?

Example

Problem:

Create a program to calculate the total price of three products.

Analysis:

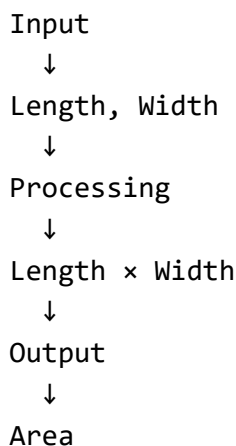
Component	Description
Input	Price 1, Price 2, Price 3
Processing	Add the three prices
Output	Total price

3. Input, Processing and Output

A simple program can often be understood using the IPO model.

IPO = Input → Processing → Output

Example:



This model helps programmers organize their thinking before writing code.

4. Opening Python Development Environment

A Python program can be written using:

- Python IDLE
- Visual Studio Code
- PyCharm
- Other approved Python IDEs/code editors

For this learning outcome, trainees may use Python IDLE or another development environment provided by the training institution.

5. Basic Structure of a Python Program

A simple Python program may contain:

```
# Program description
```

```
name = "Dara"
age = 20
```

```
print("Name:", name)
print("Age:", age)
```

The program contains:

1. Comments
2. Variables
3. Statements
4. Output instructions

6. Python Statements

A statement is an instruction that Python can execute.

Example:

```
name = "Dara"
```

Another statement:

```
print(name)
```

Python executes the statements according to the program structure.

7. Python Variables

A variable is a name used to store data.

Example:

```
name = "Dara"
age = 20
score = 85.5
```

Here:

- name stores text.
- age stores an integer.
- score stores a decimal value.

Variables allow programs to store and use information.

8. The print() Function

The print() function displays information.

Example:

```
print("Hello Python")
```

Output:

Hello Python

Variables can also be displayed:

```
name = "Dara"
print(name)
```

Multiple values can be displayed:

```
name = "Dara"
age = 20

print("Name:", name)
print("Age:", age)
```

Output:

```
Name: Dara
Age: 20
```

9. Python Comments

Comments are notes written in a program to explain the code.

A single-line comment begins with:

```
#
```

Example:

```
# Display student information
print("Student Information")
```

Comments are not executed as normal Python instructions.

Good comments can help programmers:

- Understand the purpose of code.
- Maintain programs.
- Explain important operations.
- Work with other programmers.

10. Python Indentation

Python uses indentation to define code blocks.

Example:

```
age = 20

if age >= 18:
    print("Adult")
```

The print() statement is indented.

Incorrect indentation can produce an error.

Example:

```
age = 20

if age >= 18:
print("Adult")
```

This structure is invalid because the statement inside the if block is not properly indented.

At this stage, trainees must develop the habit of using consistent indentation.

11. Python Syntax Rules

Python syntax includes rules such as:

- Correct spelling of keywords and functions.
- Correct use of brackets.
- Correct use of quotation marks.
- Correct indentation.
- Correct variable assignment.
- Correct file extension.
- Correct structure of statements.

Example:

Correct:

```
print("Hello")
```

Incorrect:

```
Print("Hello")
```

Python is case-sensitive.

print and Print are different identifiers.

12. Python Shell and Script Mode

Python Shell

Python Shell allows the programmer to enter a command and immediately see the result.

Example:

```
>>> 5 + 10
15
```

Script Mode

Script Mode stores the program in a .py file.

Example:

```
simple_program.py
```

The program can then be executed repeatedly.

For developing complete programs, Script Mode is generally more appropriate.

13. Writing a Simple Python Program

Example problem:

Write a program that displays a student's name, age and course.

Program:

```
# Student information program

name = "Dara"
age = 20
course = "AI Basic Python Developer"

print("Student Information")
print("-----")
print("Name:", name)
print("Age:", age)
print("Course:", course)
```

14. Saving a Python Program

Python source files should normally use the:

.py
extension.

Example:

```
student_information.py
```

Good file naming practices include:

- Use meaningful names.
- Avoid unnecessary spaces.
- Use .py.
- Keep related programs in organized folders.

Example folder:

```
Python_Training
|
|— L02
|   |— task01.py
```

```
|  └─ task02.py
|  └─ student_information.py
```

15. Executing a Python Program

A Python program can be executed from an IDE or command line.

For example:

```
python student_information.py
```

The program should produce the expected output.

16. Program Testing

Testing means executing a program to determine whether it produces the expected result.

A programmer should:

1. Run the program.
2. Provide appropriate input.
3. Observe the output.
4. Compare the output with the expected result.
5. Identify errors.
6. Correct errors.
7. Test again.

17. Types of Common Python Errors

Three important types of errors are introduced in this learning outcome.

17.1 Syntax Error

A syntax error occurs when Python code does not follow the language syntax rules.

Example:

```
print("Hello"
```

The closing parenthesis is missing.

17.2 Runtime Error

A runtime error occurs while the program is executing.

Example:

```
number = 10
result = number / 0
```

The program attempts to divide by zero.

17.3 Logical Error

A logical error occurs when the program runs but produces an incorrect result.

Example:

```
length = 10
width = 5

area = length + width
print(area)
```

The program runs, but the calculation is incorrect.

Correct:

```
area = length * width
```

18. Debugging

Debugging means identifying and correcting errors in a program.

A simple debugging process is:



Figure 1: Simple Debugging

19. Debugging Techniques

Trainees can use the following techniques:

Read the Error Message

Python often provides information about where an error occurred.

Check the Line Number

Look at the line identified by Python.

Check Syntax

Check:

- Parentheses
- Quotation marks
- Colons
- Indentation
- Variable names

Check Logic

Ask:

Is the program doing the correct calculation or operation?

Test Small Sections

When appropriate, test smaller parts of the program.

Use print()

Temporary output can help identify values.

Example:

```
price = 100
quantity = 5

print("Price:", price)
print("Quantity:", quantity)

total = price * quantity

print("Total:", total)
```

20. Program Verification

Verification means confirming that the program satisfies the requirements.

Example requirement:

Calculate the area of a rectangle.

Test data:

```
Length = 10
Width = 5
```

Expected result:

```
Area = 50
```

If the program displays:

```
Area = 50
```

the result is correct for that test case.

21. Good Programming Practices

Trainees should:

- Use meaningful variable names.
- Use correct indentation.
- Add useful comments.
- Keep programs organized.
- Save files using .py.
- Test programs before submission.
- Correct errors before final submission.
- Keep backup copies where appropriate.
- Follow the required programming conventions.

SELF-CHECK No. 2.1

Instructions

Answer all questions.

Part A: Multiple Choice

1. What should a programmer do before writing code?

- A. Print the program
- B. Analyze the problem
- C. Delete the program
- D. Install a printer

2. What does IPO stand for?

- A. Input, Processing, Output
- B. Internet, Program, Operating System
- C. Input, Program, Operation
- D. Installation, Processing, Output

3. Which function displays output in Python?

- A. display()
- B. show()
- C. print()
- D. output()

4. Which extension is normally used for Python source files?

- A. .txt
- B. .java
- C. .py
- D. .cpp

5. Which symbol is used to start a Python comment?

- A. //
- B. #
- C. /*
- D. --

6. What does indentation help define in Python?

- A. File size
- B. Code blocks
- C. Computer memory
- D. File extension

7. Which error occurs when Python syntax is incorrect?

- A. Logical error
- B. Syntax error
- C. Network error
- D. Hardware error

8. A program runs but produces an incorrect calculation. What type of error is this?

- A. Syntax error
- B. Logical error
- C. Installation error
- D. File error

9. Which activity is part of debugging?

- A. Ignoring errors
- B. Identifying and correcting errors
- C. Deleting Python
- D. Formatting the computer

10. Which mode is normally suitable for saving a complete Python program?

- A. Script Mode
- B. Calculator Mode
- C. Printer Mode
- D. Browser Mode

ANSWER KEY No. 2.1

Part A

No.	Answer
1	B
2	A
3	C
4	C
5	B
6	B
7	B
8	B
9	B
10	A

OPERATION SHEET No. 2.1

Title: Write, Save, Execute and Debug a Simple Python Program

Objective

Upon completion of this Operation Sheet, the trainee must be able to:

- Open a Python development environment.
- Create a Python source file.
- Write a simple Python program.
- Apply correct syntax and indentation.
- Save a .py file.
- Execute a Python program.
- Identify common errors.
- Correct simple syntax and logical errors.
- Verify the program output.

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDLE/IDE	Software	1
4	Keyboard and Mouse	Set	1
5	Command Prompt/Terminal	Software	1
6	Programming Reference Guide	Copy	1
7	CBLM	Copy	1

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze the problem	Input, processing and output are identified
2	Open Python IDE	Appropriate development environment is opened
3	Create new Python file	New source file is created
4	Write program	Program follows requirements
5	Apply indentation	Code is correctly indented
6	Add comments	Appropriate documentation is included
7	Save file	.py extension is used
8	Execute program	Program runs
9	Observe output	Output is checked
10	Identify errors	Error type/location is identified
11	Correct errors	Appropriate correction is applied
12	Test again	Program is executed again
13	Verify result	Output meets requirements

Demonstration Program

Problem:

Develop a program that calculates the total cost of purchasing a product.

Given:

Product price = 15

Quantity = 4

Formula:

Total = Price × Quantity

Python program:

```
# Calculate total product cost
```

```
price = 15
```

```
quantity = 4
```

```
total = price * quantity
```

```
print("Product Price:", price)
```

```
print("Quantity:", quantity)
```

```
print("Total Cost:", total)
```

Expected output:

```
Product Price: 15
```

```
Quantity: 4
```

```
Total Cost: 60
```

Debugging Demonstration

Error Example 1 — Syntax Error

Incorrect:

```
print("Hello"
```

Correction:

```
print("Hello")
```

Error Example 2 — Logical Error

Incorrect:

```
price = 15
```

```
quantity = 4
```

```
total = price + quantity
```

Correction:

```
total = price * quantity
```

TASK SHEET No. 2.1

Title: Analyze a Simple Programming Problem

Performance Task:

The trainee shall analyze a given programming problem and identify its input, processing and output before writing Python code.

Performance Objective:

Given a simple programming problem and worksheet, the trainee must correctly identify the required input, processing and output.

Description:

Problem:

A shop needs a program to calculate the total price of two products. The program must display the price of each product and the total price.

Picture/Overview:

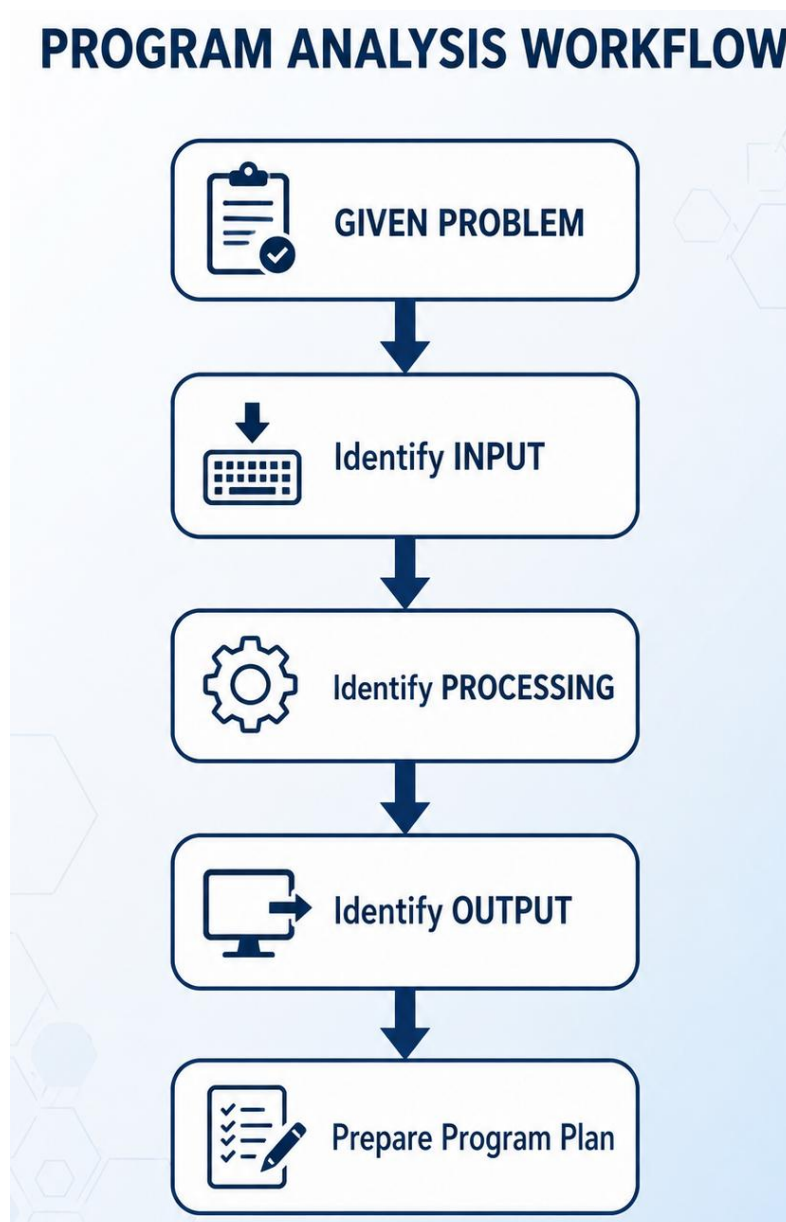


Figure 2: Problem Analysis Flow

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Worksheet	Copy	1
3	CBLM	Copy	1
4	Pen/Pencil	Unit	1

Supplier:

- Training institution

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Read the problem	Problem is understood
2	Identify required data	Input is correctly identified
3	Identify operation	Processing is correctly identified
4	Identify result	Output is correctly identified
5	Write the solution plan	Plan is logically organized

Trainee Worksheet

Input:

Processing:

Formula/Operation:

Assessment Method:

- Direct observation
- Review of completed worksheet
- Oral questioning

TASK SHEET No. 2.2

Title: Develop a Simple Student Information Program

Performance Task:

The trainee shall develop a Python program that displays student information.

Performance Objective:

Given Python IDLE or another approved Python IDE, the trainee must create, save and execute a Python program using variables, comments and print().

Description:

Create a program that displays:

- Student name
- Age
- Gender
- Course
- Level

Picture/Overview:

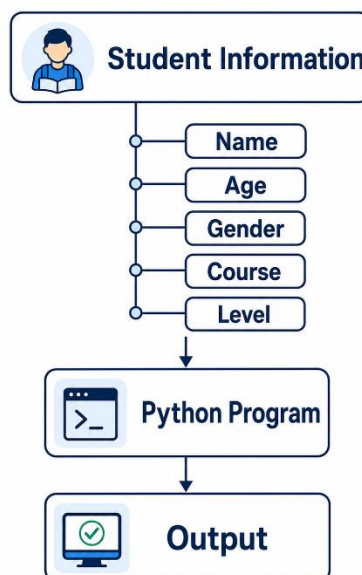


Figure 3: Task Sheet 2.2 Problem

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDLE/IDE	Software	1
4	CBLM	Copy	1
5	Worksheet	Copy	1

Supplier:

- Training institution
- Authorized software provider

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze requirements	Required information is identified
2	Open Python IDE	IDE opens correctly
3	Create file	New Python file is created
4	Declare variables	Variables store correct information
5	Add comment	Program is documented
6	Use print()	Required information is displayed
7	Save file	File uses .py extension
8	Execute program	Program runs successfully
9	Check output	Output matches requirements

Program Requirement

Create:

```
student_information.py
```

The program should produce information similar to:

```
Student Information
-----
Name: [Student Name]
Age: [Age]
Gender: [Gender]
Course: AI Basic Python Developer
Level: II
```

Assessment Method:

- Practical observation
- Source-code inspection
- Output verification
- Performance checklist

TASK SHEET No. 2.3

Title: Identify and Correct Python Errors

Performance Task:

The trainee shall identify, debug and correct syntax, runtime and logical errors in simple Python programs.

Performance Objective:

Given three faulty Python programs, the trainee must identify the error and make the required corrections.

Description:

The trainee will work with intentionally incorrect programs and use debugging techniques to correct them.

Picture/Overview:

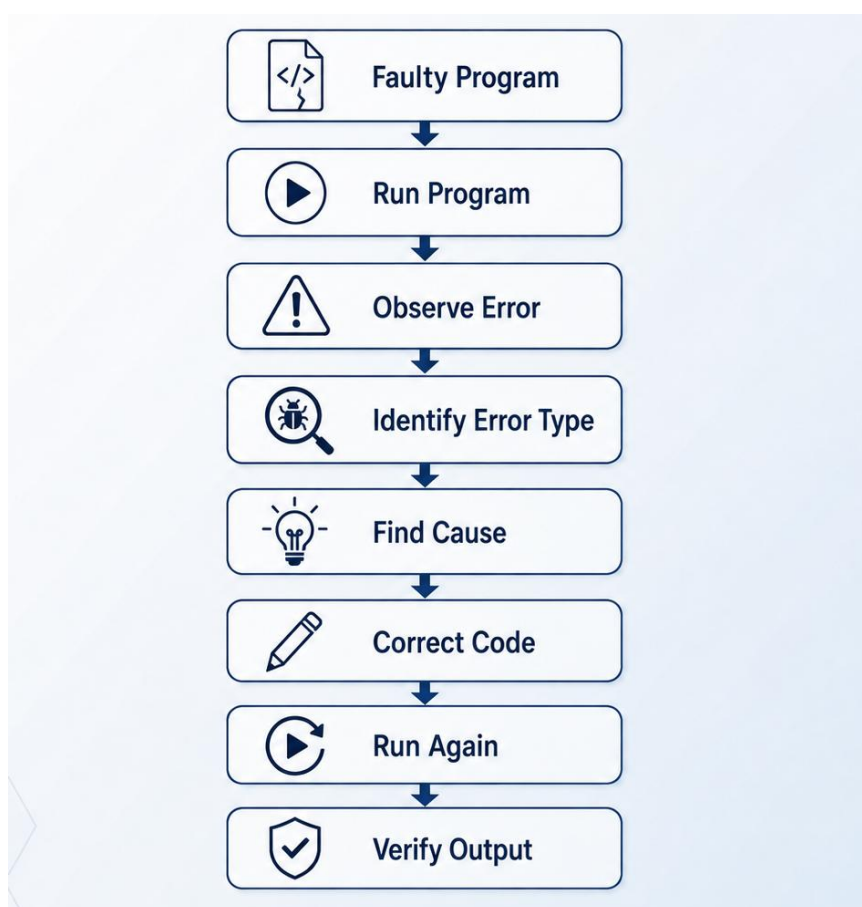


Figure 4: Task Sheet 2.3 Problem Analysis

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Error Exercise Files	Set	1
5	CBLM	Copy	1

Supplier:

- Training institution

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open faulty program	File opens successfully
2	Run program	Error is observed
3	Read error information	Error location is identified
4	Identify error type	Error is correctly classified
5	Find cause	Cause is correctly identified
6	Correct code	Appropriate correction is made
7	Run program again	Program executes
8	Verify output	Output is correct

Exercise 1 — Syntax Error

Correct:

```
name = "Dara"  
print(name)
```

Exercise 2 — Runtime Error

Correct:

```
number = 20  
result = number / 0  
print(result)
```

Exercise 3 — Logical Error

Correct:

```
length = 10  
width = 5  
  
area = length + width  
  
print("Area:", area)
```

Assessment Method:

- Direct observation
- Practical demonstration
- Error identification
- Corrected source-code inspection
- Output verification

TASK SHEET No. 2.4

Title: Develop a Simple Calculation Program

Performance Task:

The trainee shall develop a Python program to calculate the total cost and verify the result.

Performance Objective:

Given a programming problem, Python IDE and computer, the trainee must analyze the problem, develop a Python program, save it, execute it, test it and verify the output.

Description:

Develop a program for a small shop.

The program must calculate:

$$\text{Total Cost} = \text{Product Price} \times \text{Quantity}$$

Use:

Product Price = 25

Quantity = 6

Picture/Overview:

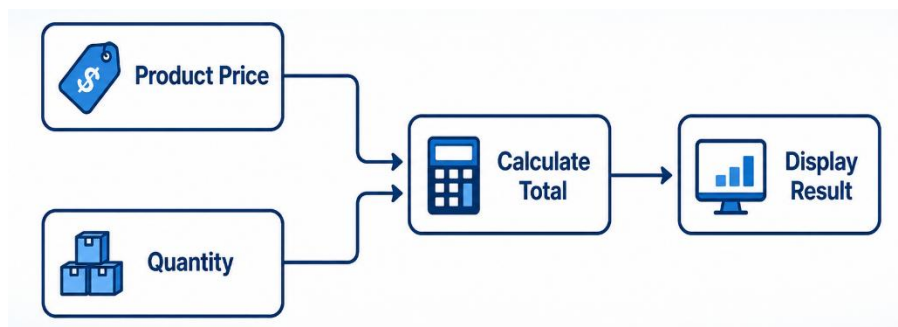


Figure 5: Task Sheet 4.1

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDLE/IDE	Software	1
4	CBLM	Copy	1
5	Worksheet	Copy	1

Supplier:

- Training institution

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Read the problem	Requirements are understood
2	Identify input	Price and quantity identified
3	Identify processing	Multiplication identified
4	Identify output	Total cost identified
5	Open Python IDE	Development environment opens
6	Create .py file	File is created correctly

No.	Procedure/Step	Key Performance
7	Write program	Program follows requirements
8	Save program	File is saved correctly
9	Execute program	Program runs
10	Test program	Test data is applied
11	Verify output	Correct result is confirmed

Expected Result

Product Price: 25

Quantity: 6

Total Cost: 150

Assessment Method:

- Direct observation
- Program source inspection
- Execution test
- Output verification
- Performance checklist

JOB SHEET No. 2.1

Title: Develop, Test and Debug a Simple Student Grade Program

Performance Task:

The trainee shall independently analyze a problem and develop a simple Python program that calculates a student's total and average marks.

Performance Objective:

Given a computer, Python IDE, programming reference guide and CBLM, the trainee must develop a working Python program that:

1. Analyzes the problem.
2. Uses an appropriate Python development environment.
3. Creates a Python program.
4. Saves the program as a .py file.
5. Executes the program.
6. Tests the program.
7. Identifies and corrects errors.
8. Verifies the final output.

Description:

A training institution needs a simple program to calculate a student's total marks and average mark.

The student has three subject scores.

Use:

Python = 80

Database = 75

Programming = 85

The program must calculate:

$\text{Total} = \text{Python} + \text{Database} + \text{Programming}$

and:

$\text{Average} = \text{Total} / 3$

The program must display:

- Student name
- Python score
- Database score
- Programming score
- Total
- Average

Picture/Overview:

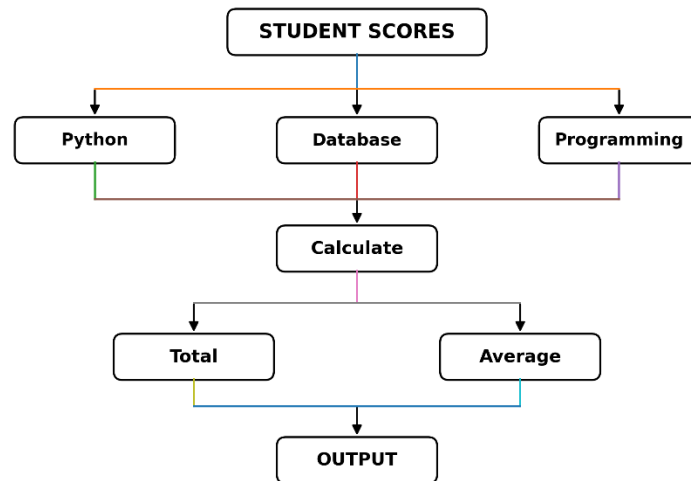


Figure 6: Job Sheet Flow

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDLE/IDE	Software	1
4	Keyboard and Mouse	Set	1
5	Command Prompt/Terminal	Software	1
6	Programming Reference Guide	Copy	1
7	CBLM	Copy	1
8	Worksheet	Copy	1
9	USB Flash Drive	Unit	1

Supplier:

- Training institution
- Authorized software provider

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze the problem	Requirements are clearly identified
2	Identify inputs	Three subject scores are identified
3	Identify processing	Total and average formulas are identified
4	Identify outputs	Required output is identified
5	Open Python IDE	Appropriate development environment is opened
6	Create source file	.py file is created
7	Write comments	Program purpose is documented
8	Create variables	Variables are appropriately named
9	Calculate total	Correct formula is used
10	Calculate average	Correct formula is used

No.	Procedure/Step	Key Performance
11	Display output	print() is used correctly
12	Save program	File is saved using .py extension
13	Execute program	Program runs successfully
14	Test program	Given test data is used
15	Check output	Output is compared with expected result
16	Identify errors	Errors are correctly identified
17	Debug program	Errors are corrected
18	Run final program	Final program executes successfully
19	Verify result	Result meets requirements
20	Organize files	Program is stored appropriately
21	Apply safety practices	Safe computer practices are followed

Programming Requirements

Create a file named:

student_grade_program.py

The program should contain appropriate comments and variables.

Expected Output

Student Grade Report

Student Name: [Your Name]

Python: 80

Database: 75

Programming: 85

Total: 240

Average: 80.0

Required Testing

The trainee must test the program using at least the following data:

Test Case 1

Python = 80

Database = 75

Programming = 85

Expected:

Total = 240

Average = 80

Test Case 2

Python = 60

Database = 70

Programming = 80

Expected:

Total = 210

Average = 70

Test Case 3

Use a different set of valid scores selected by the trainee.

Record:

Python: _____

Database: _____

Programming: _____

Expected Total: _____

Actual Total: _____

Expected Average: _____

Actual Average: _____

Debugging Requirement

The trainer may provide a version of the program containing:

- One syntax error
- One logical error

The trainee must:

1. Execute the program.
2. Identify the error.
3. Explain the cause.
4. Correct the program.
5. Execute it again.
6. Verify the output.

Final Submission Requirements

The trainee must submit:

1. Python source file.
2. Correct .py extension.
3. Working program.
4. Test results.
5. Corrected version after debugging.
6. Completed worksheet.
7. Completed performance checklist.

Assessment Method:

- Oral questioning
- Written/theoretical assessment
- Practical workshop assessment
- Direct observation
- Source-code inspection
- Program execution
- Output verification
- Debugging demonstration

PERFORMANCE CHECKLIST No. 2.1

Task: Develop, Test and Debug a Simple Python Program

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Analyzed the given programming problem	☐	☐
2	Identified the required input	☐	☐
3	Identified the required processing	☐	☐
4	Identified the required output	☐	☐
5	Opened Python IDLE or appropriate Python development environment	☐	☐
6	Created a new Python source file	☐	☐
7	Applied correct Python syntax	☐	☐
8	Applied correct indentation	☐	☐
9	Used appropriate variable names	☐	☐
10	Used comments appropriately	☐	☐
11	Used print() correctly	☐	☐
12	Developed the program according to requirements	☐	☐
13	Saved the program with .py extension	☐	☐
14	Organized the Python source file appropriately	☐	☐
15	Executed the Python program successfully	☐	☐
16	Tested the program using appropriate test data	☐	☐
17	Identified syntax errors correctly	☐	☐
18	Identified runtime errors correctly	☐	☐
19	Identified logical errors correctly	☐	☐
20	Applied appropriate debugging techniques	☐	☐
21	Corrected identified errors	☐	☐
22	Re-executed the corrected program	☐	☐
23	Verified the program output	☐	☐
24	Compared actual results with expected results	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Trainer's Name: _____

Trainer's Signature: _____

Date: _____